ELO320 - Estructuras de Datos y Algoritmos
Ordenamiento en Tiempo Lineal
Radix Sort

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

2do Semestre 2022

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Pregunta

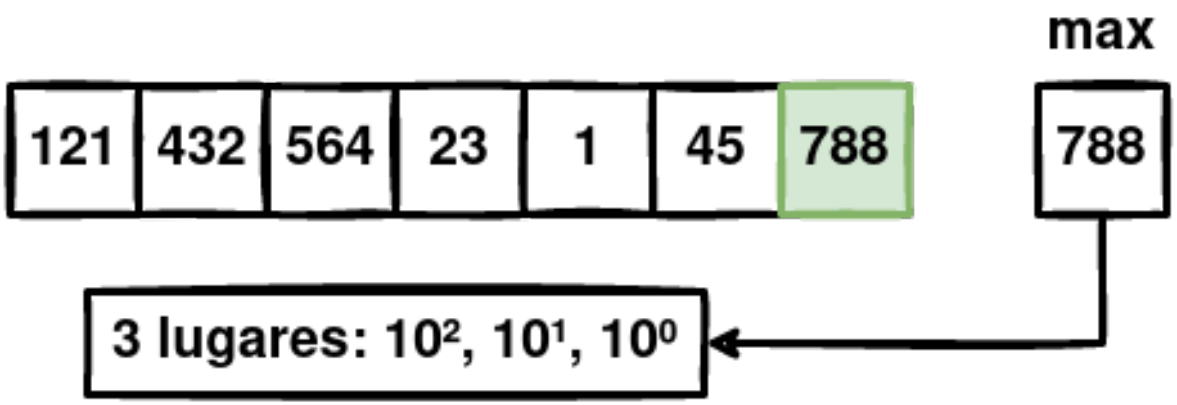
- Para ordenar enteros, ¿Es siempre mejor Counting Sort que algoritmos de Ordenamiento por Comparación?
- Si le piden ordenar un arreglo con numeros del 1 a n^3 ¿Usaría ud. Counting Sort como algoritmo? Justifique.

Radix Sort

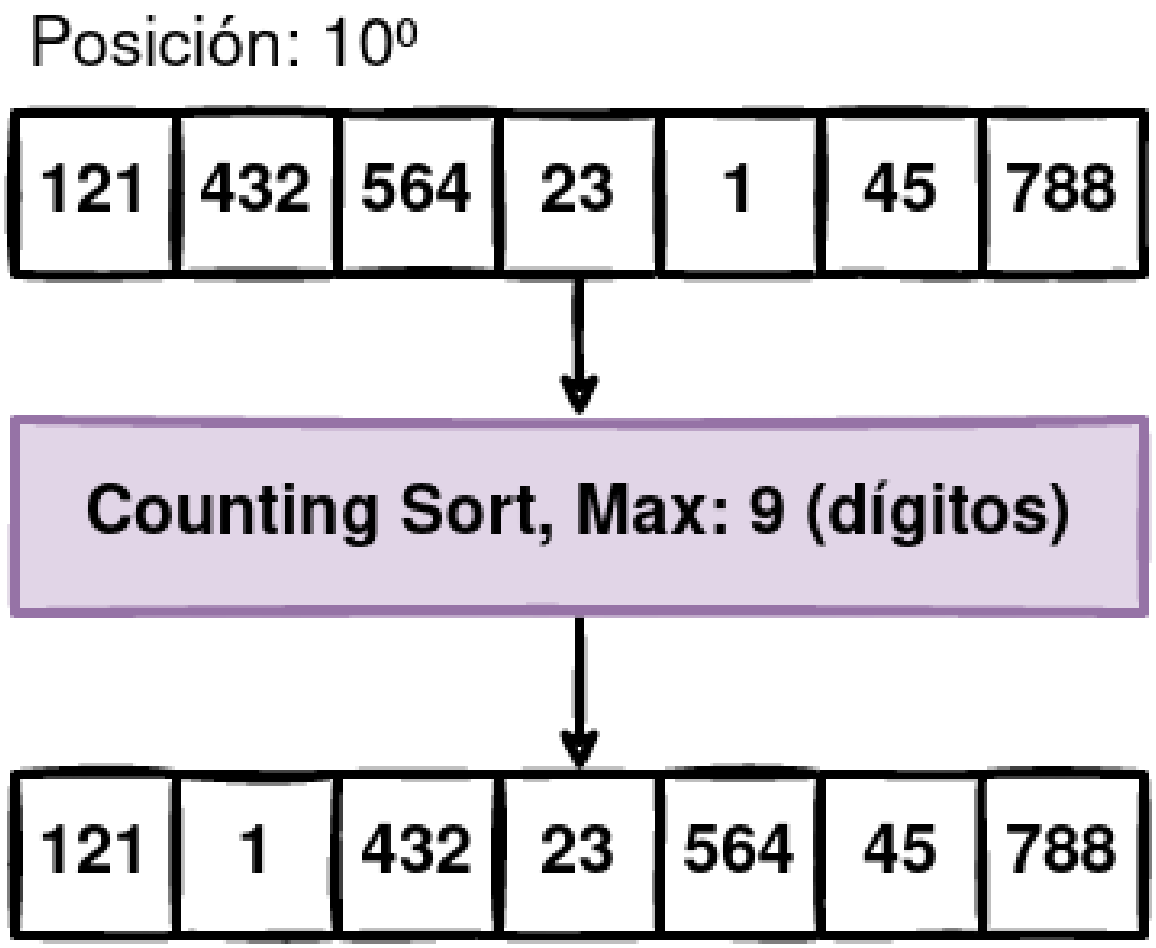
Algoritmo de Ordenamiento basado en dígitos, para poder ordenar desde las unidades hacia adelante, los elementos en un arreglo.

- Complejidad: $\mathcal{O}(d \times (n + k))$, con d la cantidad de digitos usados, n numero de elementos del arreglo y k el máximo elemento del arreglo.
- Suele ser usado para enteros, y representaciones discretas. ($k = 9 =$cte.)
- Por lo tanto, se suele decir que es $\mathcal{O}(d \times n)$.
- Se basa en una algoritmo estable para cada etapa $\rightarrow$ Counting Sort.
- Utiliza bastante memoria extra.

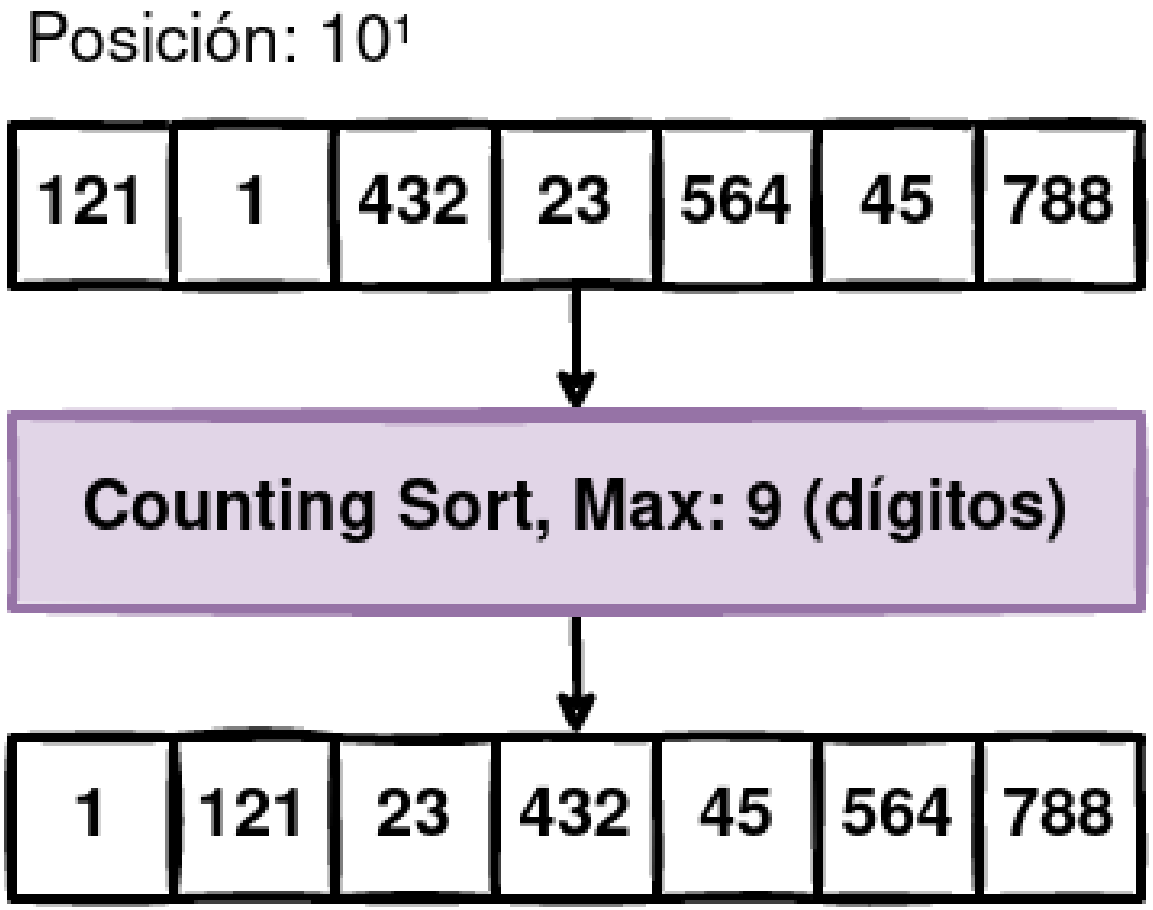
Radix Sort: Idea General



Radix Sort: Idea General

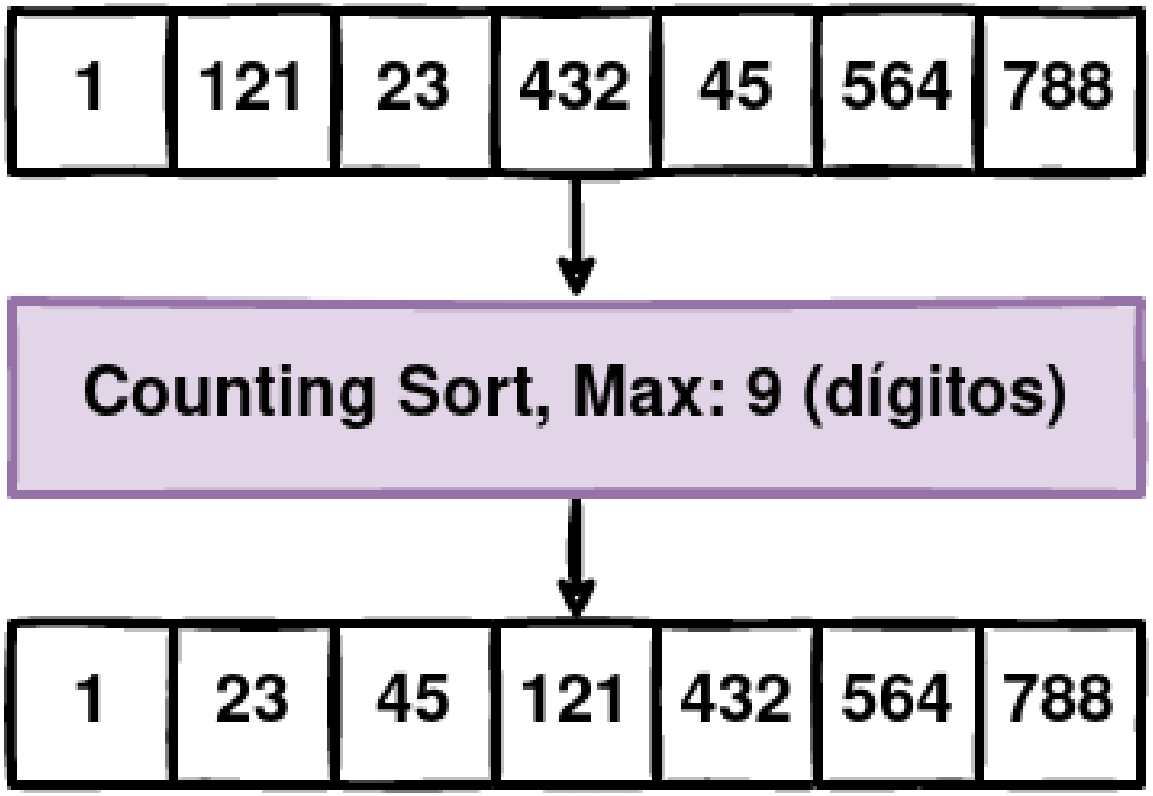


Radix Sort: Idea General



Radix Sort: Idea General

Posición: 10^2



Radix Sort

```
int maximo(int *A, int tam) {
        int i;
        int mayor = A[0];
        for (i = 1; i < tam; i++)
                if (A[i] > mayor)
                        mayor = A[i];
        return mayor;
}

void RadixSort(int *A, int tam) {
        int pos;
        int max = maximo(A, tam);

        for (pos=0 ; (max/ pow(10,pos)) > 0; pos++)
                CountingSort(A, tam, pow(10,pos));
}
```

Radix Sort: built-in Counting Sort

```
void CountingSort(int *A, int tam, int lugar) {
        int B[tam], i, ind_sgte, veces;
        int max = (A[0] / lugar) % 10; //por si no hay digitos finales

        for (i = 1; i < tam; i++) {
                if (((A[i] / lugar) % 10) > max)
                        max = (A[i]/lugar)% 10;
        }

        int C[max + 1];

        for (int i = 0; i < max; ++i)
                C[i] = 0;

        for (int i = 0; i < tam; i++)
                C[(A[i] / lugar) % 10]++;

        ind_sgte = 0;
        for(i=0;i<tam;i++){ //indice siguiente
                veces=C[i];
                C[i] = ind_sgte;
                ind_sgte += veces;
        }

        for(i=0;i<tam;i++){ //paso a B
                B[C[(A[i]/lugar) % 10]] = A[i];
                C[(A[i]/lugar) % 10]++;
        }

}
```

Ejercicio

- Utilizando RadixSort, ordene lexicográficamente la siguiente lista de palabras en inglés: cow, dog, sea, rug, row, mob, box, tab, bar, ear, tar, dig, big, tea, now, fox.